The difficulty with databases is that they contain data, so blowing it away and creating a new database with every deployment isn't possible. A common best practice is to version a database's schema explicitly along with your application's code. You can do this in many ways, but typically you need to store the SQL script that takes the database from the previous schema to the new schema. Then you can use a library such as DbUp (https://github.com/DbUp/DbUp) or FluentMigrator (https://github.com/fluentmigrator/fluentmigrator) to keep track of which scripts have been applied and ensure that your database schema is up to date. Alternatively, you can use external tools to manage this task.

EF Core provides its own version of schema management called *migrations*. Migrations provide a way to manage changes to a database schema when your EF Core data model changes.

DEFINITION A *migration* is a C# code file in your application that defines how the data model changed—which columns were added, new entities, and so on. Migrations provide a record over time of how your database schema evolved as part of your application, so the schema is always in sync with your app's data model.

You can use command-line tools to create a new database from the migrations or to update an existing database by applying new migrations to it. You can even roll back a migration, which updates a database to a previous schema.

WARNING Applying migrations modifies the database, so you must always be aware of data loss. If you remove a table from the database using a migration and then roll back the migration, the table will be re-created, but the data it previously contained will be gone forever!